Exercício em duplas:

Para cada item: **(a)** o que o Prolog responde? **(b)** qual regra explica?

1 · Unificação

```
?- f(X, b) = f(a, Y).

?- f(X, X) = f(a, b).
```

(a) o que o Prolog responde?
R: X = a, Y = b e o segundo com false.

(b) qual regra explica?
R: A Unificação casa os dois termos posição por posição. X com a e Y com b. No caso do segundo, X teria que ser a e b ao mesmo tempo, o que gera um conflito.

2 · Aritmética

```
?- X = 2 + 3.

?- X is 2 + 3.

?- 5 = 2 + 3.
```

(a) o que o Prolog responde?
R: X = 2 + 3, X = 5, false

(b) qual regra explica?

R: O primeiro é a Unificação, sem cálculo, apenas junta. O segundo “is” força a avaliação aritmética, realizando o cálculo. O terceiro usa a Unificação de um número (5) para tentar juntar com uma estrutura composta, e sem uma avaliação prévia, eles não se juntam.

3 · Ordem das respostas

```
cor(maca, vermelha).
cor(banana, amarela).
cor(morango, vermelha).

?- cor(F, vermelha).
```

(a) o que o Prolog responde?

R: F = maca; F = morango

(b) qual regra explica?

R: A pergunta é “existe alguma premissa onde (F, vermelha) pode ser verdadeiro?”, avalia os Fatos e lista os dois onde a condição bate.

4 · Listas

```
?- [H | T] = [a, b, c].

?- [A, B | R] = [1, 2].
```

(a) o que o Prolog responde?

R: H = a, T = [b, c] e depois A = 1, B = 2, R = []

(b) qual regra explica?

R: A cauda (argumento após |) sempre será uma lista, podendo ela ser vazia ou não. A cabeça sempre pega os argumentos de acordo com o pedido, onde o primeiro pede 1 elemento, enquanto o segundo pede dois [A, B].

5 · Negação

```
gosta(ana, prolog).

?- \+ gosta(bruno, prolog).

?- \+ gosta(X, prolog).
```

(a) o que o Prolog responde?

R: true, false

(b) qual regra explica?

R: bruno, prolog é um fato falso, e a negação dessa falsidade devolve um yes/true. Já X, prolog pergunta se a variável X pode ser substituída por um valor verdadeiro, e a resposta é sim, pois existe ana, prolog. No entanto, a negação dessa afirmação devolve um false.

6 · Ordem das cláusulas

```
ancestral(X, Y) :-
    ancestral(Z, Y), pai(X, Z).
ancestral(X, Y) :- pai(X, Y).
pai(ana, bia).

?- ancestral(ana, bia).
```

(a) o que o Prolog responde?

R: Erro de stack overflow causada por loop infinito.

(b) qual regra explica?

R: A primeira cláusula chama ANCESTRAL novamente antes de consultar qualquer fato, gerando um loop infinito.